ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Хеш-функції та хеш-таблиці

Звіт про виконання лабораторної роботи № 5

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Вінниця 2026

Тема: Дослідження хеш-функцій та хеш-таблиць. Хеш-функція SHABAL. Метод множення.

Мета: ознайомитися з відомими алгоритмами побудови хеш-функцій; реалізувати хеш-функцію SHABAL та власну хеш-функцію методом множення; застосувати їх для обчислення контрольної суми файлу і порівняти якість розподілу ключів у хеш-таблиці.

ЗАВДАННЯ

1. Обчислити хеш-функцію відомим методом. Для варіанта 1.12 — хеш-функція SHABAL.

2. Придумати та реалізувати власну хеш-функцію. Варіант 12 парний, тому застосовано метод множення.

3. Застосувати розроблені хеш-функції— обчислення контрольної суми заданого файлу, яка дописується в кінець цього файлу.

4. Передбачити в програмі зручні методи роботи користувача (меню з вибором дії).

5. Порівняти хеш-функції: створити хеш-таблицю на 10^3–10^4 значень, заповнити її на 50–70 % і порахувати кількість колізій для 500–1000 ключів; врахувати вплив діапазону вхідних даних.

КОД ПРОГРАМИ

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <vector>
#include <string>
#include <random>
#include <iomanip>
#include <unordered_set>
#include <cstdint>

using namespace std;

// ======================== SHABAL-512 ========================

uint32_t rotl(uint32_t x, int n) {
    return (x << n) | (x >> (32 - n));
}

uint32_t U(uint32_t x) { return x * 3; }
uint32_t V(uint32_t x) { return x * 5; }

void encodeBlock(const uint32_t M[16], unsigned char* buf) {
    for (int i = 0; i < 16; i++)
```

```
        for (int j = 0; j < 4; j++)
            buf[4*i + j] = (unsigned char)(M[i] >> (8*j));
}

void shabalP(uint32_t A[], uint32_t B[], uint32_t C[], const uint32_t M[]) {
    for (int i = 0; i < 16; i++)
        B[i] = rotl(B[i], 17);

    for (int j = 0; j < 3; j++)
        for (int i = 0; i < 16; i++) {
            int k = (i + 4*j) % 12;
            A[k] = U(A[k] ^ C[(8 - i + 16) % 16] ^ V(rotl(A[(k + 11) % 12], 15)))
                   ^ M[i] ^ B[(i + 13) % 16] ^ (B[(i + 9) % 16] & ~B[(i + 6) % 16]);
            B[i] = ~(rotl(B[i], 1) ^ A[k]);
        }

    for (int i = 0; i < 36; i++)
        A[i % 12] += C[(i % 12 + 11 + 4*(i / 12)) % 16];
}

void shabalBlock(uint32_t A[], uint32_t B[], uint32_t C[], const uint32_t M[],
                 uint32_t& Wlow, uint32_t& Whigh) {
    for (int i = 0; i < 16; i++) B[i] += M[i];
    A[0] ^= Wlow;
    A[1] ^= Whigh;
    shabalP(A, B, C, M);
    for (int i = 0; i < 16; i++) C[i] -= M[i];
    for (int i = 0; i < 16; i++) { uint32_t t = B[i]; B[i] = C[i]; C[i] = t; }
    if (++Wlow == 0) ++Whigh;
}

void shabal512(const vector<unsigned char>& msg, uint32_t out[16]) {
    uint32_t A[12] =
{0x20728DFDu,0x46C0BD53u,0xE782B699u,0x55304632u,0x71B4EF90u,0x0EA9E82Cu,
                      0xDBB930F1u,0xFAD06B8Bu,0xBE0CAE40u,0x8BD14410u,0x76D2ADACu,0x28ACAB7F
u};
    uint32_t B[16] =
{0xC1099CB7u,0x07B385F3u,0xE7442C26u,0xCC8AD640u,0xEB6F56C7u,0x1EA81AA9u,
                      0x73B9D314u,0x1DE85D08u,0x48910A5Au,0x893B22DBu,0xC5A0DF44u,0xBBC4324E
u,
                      0x72D2F240u,0x75941D99u,0x6D8BDE82u,0xA1A7502Bu};
    uint32_t C[16] =
{0xD9BF68D1u,0x58BAD750u,0x56028CB2u,0x8134F359u,0xB5D469D8u,0x941A8CC2u,
                      0x418B2A6Eu,0x04052780u,0x7F07D787u,0x5194358Fu,0x3C60D665u,0xBE97D79A
u,
                      0x950C3434u,0xAED9A06Du,0x2537DC8Du,0x7CDB5969u};
    uint32_t Wlow = 1, Whigh = 0;
    uint32_t M[16];

    size_t pos = 0;
    while (msg.size() - pos >= 64) {
        for (int i = 0; i < 16; i++)
            M[i] = msg[pos + 4*i] | (uint32_t(msg[pos + 4*i+1]) << 8) |
                   (uint32_t(msg[pos + 4*i+2]) << 16) | (uint32_t(msg[pos + 4*i+3]) << 24);
        shabalBlock(A, B, C, M, Wlow, Whigh);
        pos += 64;
    }

    unsigned char buf[64] = {};
```

```
    for (size_t i = 0; pos + i < msg.size(); i++) buf[i] = msg[pos + i];
    buf[msg.size() - pos] = 0x80;
    for (int i = 0; i < 16; i++)
        M[i] = buf[4*i] | (uint32_t(buf[4*i+1]) << 8) |
               (uint32_t(buf[4*i+2]) << 16) | (uint32_t(buf[4*i+3]) << 24);

    for (int i = 0; i < 16; i++) B[i] += M[i];
    A[0] ^= Wlow;
    A[1] ^= Whigh;
    shabalP(A, B, C, M);

    for (int i = 0; i < 3; i++) {
        for (int k = 0; k < 16; k++) { uint32_t t = B[k]; B[k] = C[k]; C[k] = t; }
        A[0] ^= Wlow;
        A[1] ^= Whigh;
        shabalP(A, B, C, M);
    }
    for (int i = 0; i < 16; i++) out[i] = B[i];
}

// ============== ВЛАСНА ФУНКЦІЯ (==============

const uint32_t A_MUL = 2654435769u;

uint32_t mulHash(uint32_t k) {
    return (uint32_t)((uint64_t)k * A_MUL);
}

int mulIndex(uint32_t k, int bits) {
    return (int)(mulHash(k) >> (32 - bits));
}
// ----------------------------------------
vector<unsigned char> readFile(const string& name, bool& ok) {
    ifstream f(name, ios::binary);
    ok = (bool)f;
    if (!ok) return {};
    vector<unsigned char> d;
    char buf[4096];
    while (f.read(buf, 4096) || f.gcount())
        d.insert(d.end(), buf, buf + f.gcount());
    return d;
}

void calcChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }

    uint32_t h[16];
    shabal512(data, h);

    uint32_t k = 0;
    for (unsigned char c : data) k = k * A_MUL + c;
    uint32_t mine = mulHash(k);

    cout << "Файл: " << name << "   (" << data.size() << " байт)" << endl;
    cout << "SHABAL-512:" << endl << "  " << hex << setfill('0');
    for (int i = 0; i < 16; i++) cout << setw(8) << h[i];
    cout << dec << setfill(' ') << endl;
```

```
    cout << "Власна: " << hex << setfill('0')
         << setw(8) << mine << dec << setfill(' ') << endl;
}

void appendChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }

    uint32_t h[16];
    shabal512(data, h);
    unsigned char sum[64];
    encodeBlock(h, sum);

    ofstream f(name, ios::binary | ios::app);
    f.write((char*)sum, 64);
    cout << "Контрольну суму (64 байти) дописано в кінець файлу " << name << endl;
}

void verifyChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }
    if (data.size() < 64) { cout << "Файл закороткий, суми в ньому немає." << endl; return;
}

    size_t n = data.size() - 64;
    uint32_t h[16];
    shabal512(vector<unsigned char>(data.begin(), data.begin() + n), h);

    uint32_t saved[16];
    for (int i = 0; i < 16; i++)
        saved[i] = data[n + 4*i] | (uint32_t(data[n + 4*i+1]) << 8) |
                   (uint32_t(data[n + 4*i+2]) << 16) | (uint32_t(data[n + 4*i+3]) << 24);

    bool same = true;
    for (int i = 0; i < 16; i++)
        if (saved[i] != h[i]) same = false;
    cout << (same ? "Positive"
                  : "NEGATIVE!") << endl;
}

vector<uint32_t> makeKeys(uint32_t range, int count, mt19937& gen) {
    vector<uint32_t> keys;
    unordered_set<uint32_t> used;
    uniform_int_distribution<uint32_t> dist(0, range - 1);
    while ((int)keys.size() < count) {
        uint32_t k = dist(gen);
        if (used.insert(k).second) keys.push_back(k);
    }
    return keys;
}

int shabalIndexOf(uint32_t key, int bits) {
    unsigned char b[4];
    for (int j = 0; j < 4; j++) b[j] = (unsigned char)(key >> (8*j));
    vector<unsigned char> msg(b, b + 4);
    uint32_t h[16];
    shabal512(msg, h);
```

```
    return (int)(h[0] >> (32 - bits));
}

void collisions() {
    const int bits = 13;
    const int m = 1 << bits;
    const int fill = m * 60 / 100;
    const int keys = 700;
    const uint32_t ranges[3] = {10000u, 1000000u, 1000000000u};
    const char* rname[3] = {"0..10^4", "0..10^6", "0..10^9"};

    cout << "Хеш-таблиця: " << m << " комірок, зайнято " << fill
         << " (" << (fill*100 + m/2)/m << "%), додається " << keys << " нових ключів." <<
endl;
    double frac = 1.0 - exp(-(double)fill / m);
    cout << "Очікувано колізій при рівномірному хешуванні: " << (int)(keys * frac) << endl
<< endl;

    for (int r = 0; r < 3; r++) {
        mt19937 gen(1000 + r);
        vector<uint32_t> all = makeKeys(ranges[r], fill + keys, gen);
        vector<uint32_t> base(all.begin(), all.begin() + fill);
        vector<uint32_t> test(all.begin() + fill, all.end());

        cout << "Діапазон ключів " << rname[r] << ":" << endl;
        const char* names[2] = {"SHABAL-512            ", "Власна: "};
        for (int f = 0; f < 2; f++) {
            vector<char> used(m, 0);
            auto idx = [&](uint32_t k) { return f == 0 ? shabalIndexOf(k, bits)
                                                      : mulIndex(k, bits); };
            for (int i = 0; i < fill; i++) used[idx(base[i])] = 1;
            int hit = 0;
            for (int i = 0; i < keys; i++) {
                int at = idx(test[i]);
                if (used[at]) hit++;
                else used[at] = 1;
            }
            cout << "    " << names[f] << ": " << hit << " колізій з " << keys << endl;
        }
        cout << endl;
    }
}

int main() {
    string file = "getfile.pdf";
    int choice = -1;



    cout << "Файл для обробки (Enter — " << file << "): ";
    getline(cin, file);
    if (file.empty()) file = "getfile.pdf";

    while (choice != 0) {
        cout << "\n1. Контрольна сума файлу\n2. Дописати суму в кінець файлу\n"
                "3. Перевірити дописану суму\n4. Порівняння хеш-функцій\n0. Вихід\n"
                "Вибір: ";
        if (!(cin >> choice)) break;
        cin.ignore(1000, '\n');
```

```
        switch (choice) {
            case 1: calcChecksum(file); break;
            case 2: cout << "Файл буде змінено (дописування 64 байтів)!\n";
                    appendChecksum(file); break;
            case 3: verifyChecksum(file); break;
            case 4: collisions(); break;
            case 0: break;
            default: cout << "Введіть число від 0 до 4." << endl;
        }
    }
    cout << "Кінець" << endl;
    return 0;
}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Вивід програми:

```
Лабораторна робота №5. Хеш-функції та хеш-таблиці.
Варіант 1.12 — SHABAL, застосування 3б), власна функція методом множення.
Файл для обробки (Enter — getfile.pdf):
1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Файл: test_copy.pdf   (432484 байт)
SHABAL-512:

aa219a5db5a98d7020921c91608ccf559f4e41d15d58ddcb13af014be2eaea3bb673097f455e52895c9db
6aff0f72dc2ae32144705e300e0c5f5038ff83277de
Власна, метод множення (32 біт): e7ca1ed9

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Файл буде змінено (дописування 64 байтів)!
```

Контрольну суму (64 байти) дописано в кінець файлу test_copy.pdf

```

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Перевірка УСПІШНА — контрольна сума збігається.

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Хеш-таблиця: 8192 комірок, зайнято 4915 (60%), додається 700 нових ключів.
Очікувано колізій при рівномірному хешуванні: 315

Діапазон ключів 0..10^4:
    SHABAL-512            : 332 колізій з 700
    Власна, метод множення: 176 колізій з 700

Діапазон ключів 0..10^6:
    SHABAL-512            : 334 колізій з 700
    Власна, метод множення: 345 колізій з 700

Діапазон ключів 0..10^9:
    SHABAL-512            : 334 колізій з 700
    Власна, метод множення: 337 колізій з 700


1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
```

```
0. Вихід
Вибір: Кінець

```

БЛОК-СХЕМА ФУНКЦІЇ СТИСНЕННЯ SHABAL

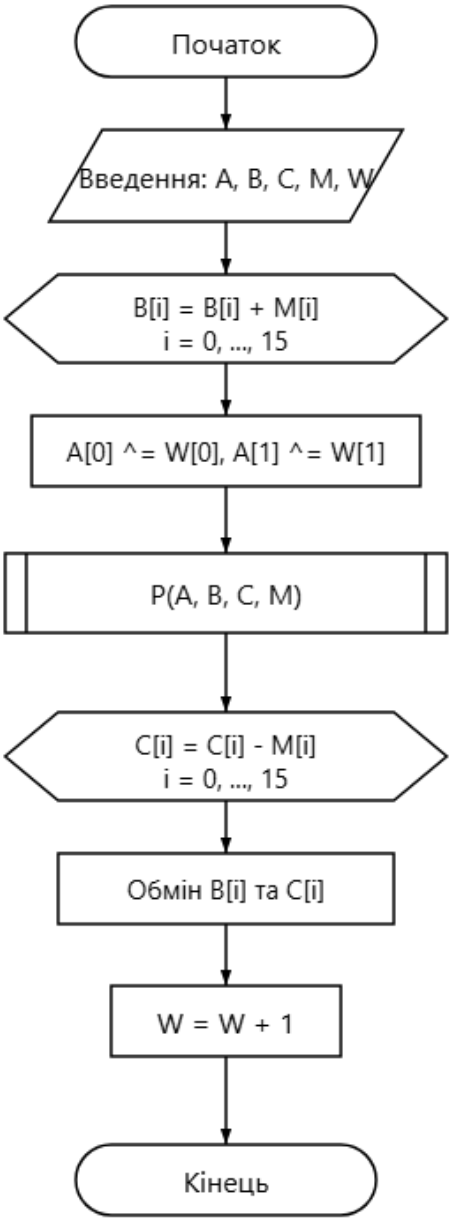


Рисунок 1 — Функція стиснення одного блоку Shabal

Вінниця 2026

БЛОК-СХЕМА ПЕРЕВІРКИ КОНТРОЛЬНОЇ СУМИ

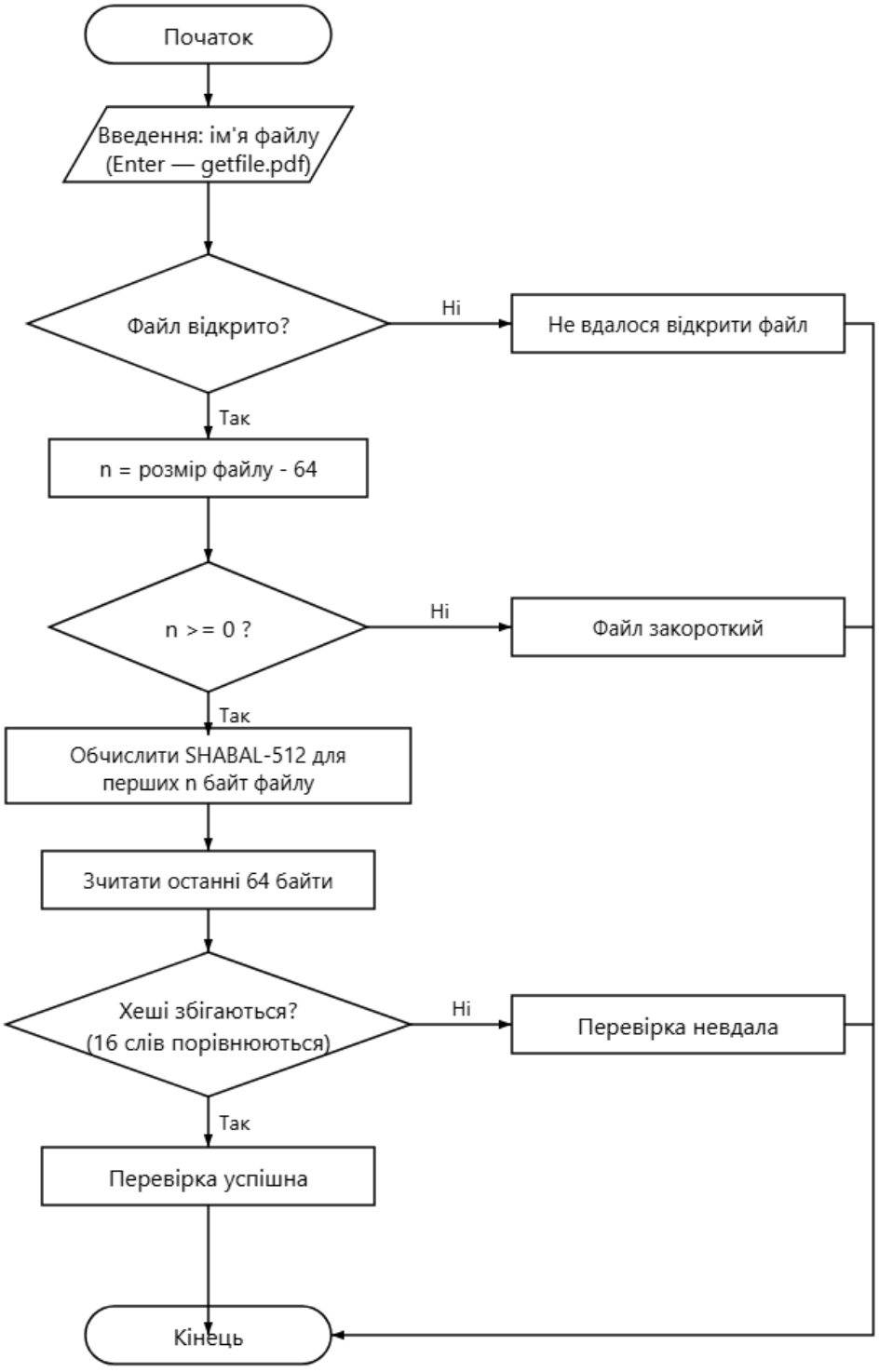


Рисунок 2 — Дописування та перевірка контрольної суми файлу

Вінниця 2026

СХЕМА РОБОТИ ВЛАСНОЇ ХЕШ-ФУНКЦІЇ

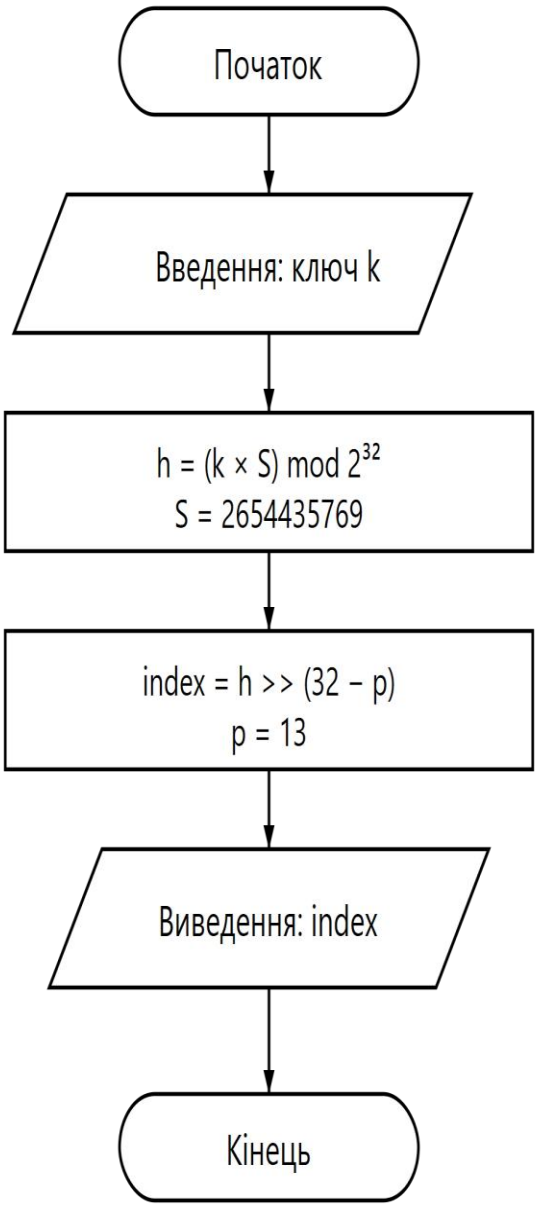


Рисунок 3 — Схема роботи власної хеш-функції (метод множення)

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи реалізовано хеш-функцію SHABAL-512 та власну хеш-функцію методом множення. Реалізація SHABAL побудована згідно зі специфікацією (перестановка P з 48 раундами, нелінійні множення U(x)=3x і V(x)=5x, фіналізація з трьома додатковими раундами) і звірена з еталонною реалізацією sphlib 3.0 на низці тестових повідомлень різної довжини (у тому числі порожнє повідомлення) — усі значення збігаються. Власна хеш-функція реалізована методом множення: h(k) — молодше 32-бітне слово добутку k·S, де S = 2654435769 = floor((sqrt(5)-1)/2 * 2^32); для довільних даних (файлу) байти згортаються в число k тим самим множенням, а номер комірки таблиці — старші p бітів h(k). Застосування 3б) реалізовано повністю: програма обчислює контрольну суму файлу, дописує її 64 байти в кінець файлу, а окремий пункт меню повторно обчислює хеш від частини файлу без суми та порівнює її із дописаними байтами; перевірка успішна, отже механізм контролю цілісності працює коректно. Експеримент з колізіями: хеш-таблиця на 8192 комірок (10^3-10^4), заповнення 60 %, додається 700 нових ключів; очікувано близько 315 колізій при рівномірному хешуванні. Результати: для діапазону 0..10^4 SHABAL-512 — 332, власна функція — 176 колізій; для 0..10^6 — 334 і 345; для 0..10^9 — 334 і 337 колізій відповідно. Основний висновок експерименту: кількість колізій визначається насамперед співвідношенням діапазону вхідних даних і розміру таблиці: при вузькому діапазоні 0..10^4 (усього у 10^4 значень на 8192 комірок) кількість колізій суттєво залежить від якості розподілу конкретної функції — проста власна функція методом множення дала помітно менше колізій (176), ніж криптографічна SHABAL-512 (332), тоді як для широких діапазонів 0..10^6 та 0..10^9 обидві функції зблизилися між собою та з очікуваним рівнем (~315-345). Реалізація використовує лише стандартні засоби мови C++, без сторонніх бібліотек.

Вінниця 2026